

本讲内容

2.1 复杂性函数阶的计算

2.2 和式的估计与界限

2.3 递归方程



增长函数

- 渐进效率:
 - 输入规模非常大
 - 忽略低阶项和常系数
 - 只考虑最高阶（增长的阶）
- 典型的增长阶:
 - $\Theta(1), \Theta(\lg n), \Theta(\sqrt{n}), \Theta(n), \Theta(n \lg n),$
 - $\Theta(n^2), \Theta(n^3), \Theta(2^n), \Theta(n!)$
- 增长的记号: $O, \Theta, \Omega, o, \omega$.

分裂求和

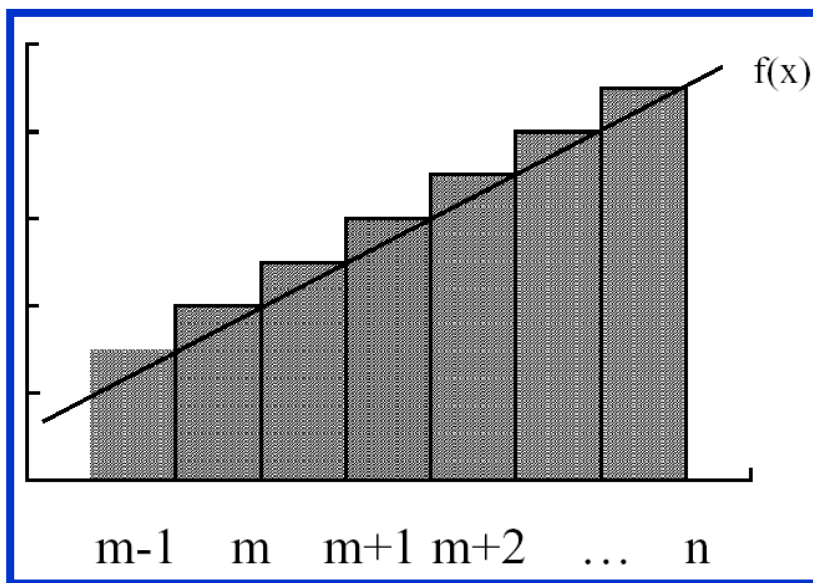
例1. 用分裂求和的方法求 $\sum_{k=1}^n k$ 的下界。

解：

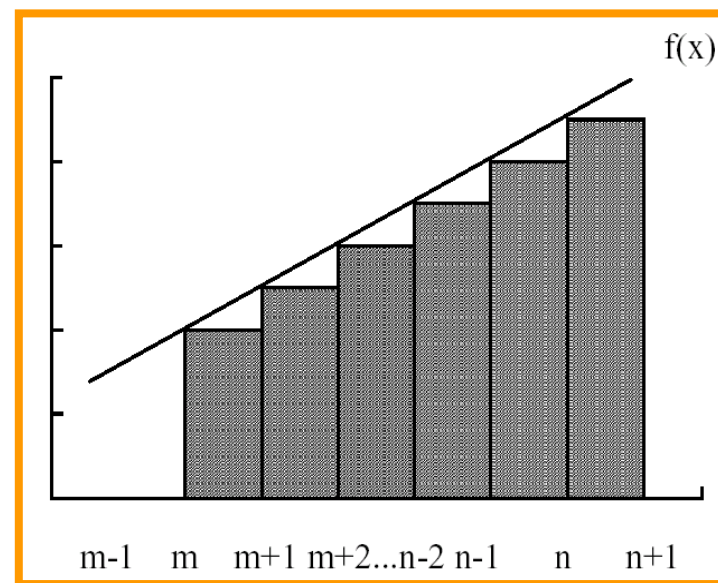
$$\sum_{k=1}^n k = \sum_{k=1}^{\lfloor n/2 \rfloor} k + \sum_{k=\lfloor n/2 \rfloor + 1}^n k \geq \sum_{k=1}^{\lfloor n/2 \rfloor} 0 + \sum_{k=\lfloor n/2 \rfloor + 1}^n \frac{n}{2} \geq \left(\frac{n}{2}\right)^2 = \Omega(n^2)$$

积分求和的近似

例1. 如果 $f(k)$ 单调递增, 则 $\int_{m-1}^n f(x)dx \leq \sum_{k=m}^n f(k) \leq \int_m^{n+1} f(x)dx$ 。



$$\sum_{k=m}^n f(k) = \sum_{k=m}^n f(k)\Delta x \geq \int_{m-1}^n f(x)dx, f(m-1) < f(n), \Delta x = 1$$



$$\sum_{k=m}^n f(k) = \sum_{k=m}^n f(k)\Delta x \leq \int_m^{n+1} f(x)dx$$

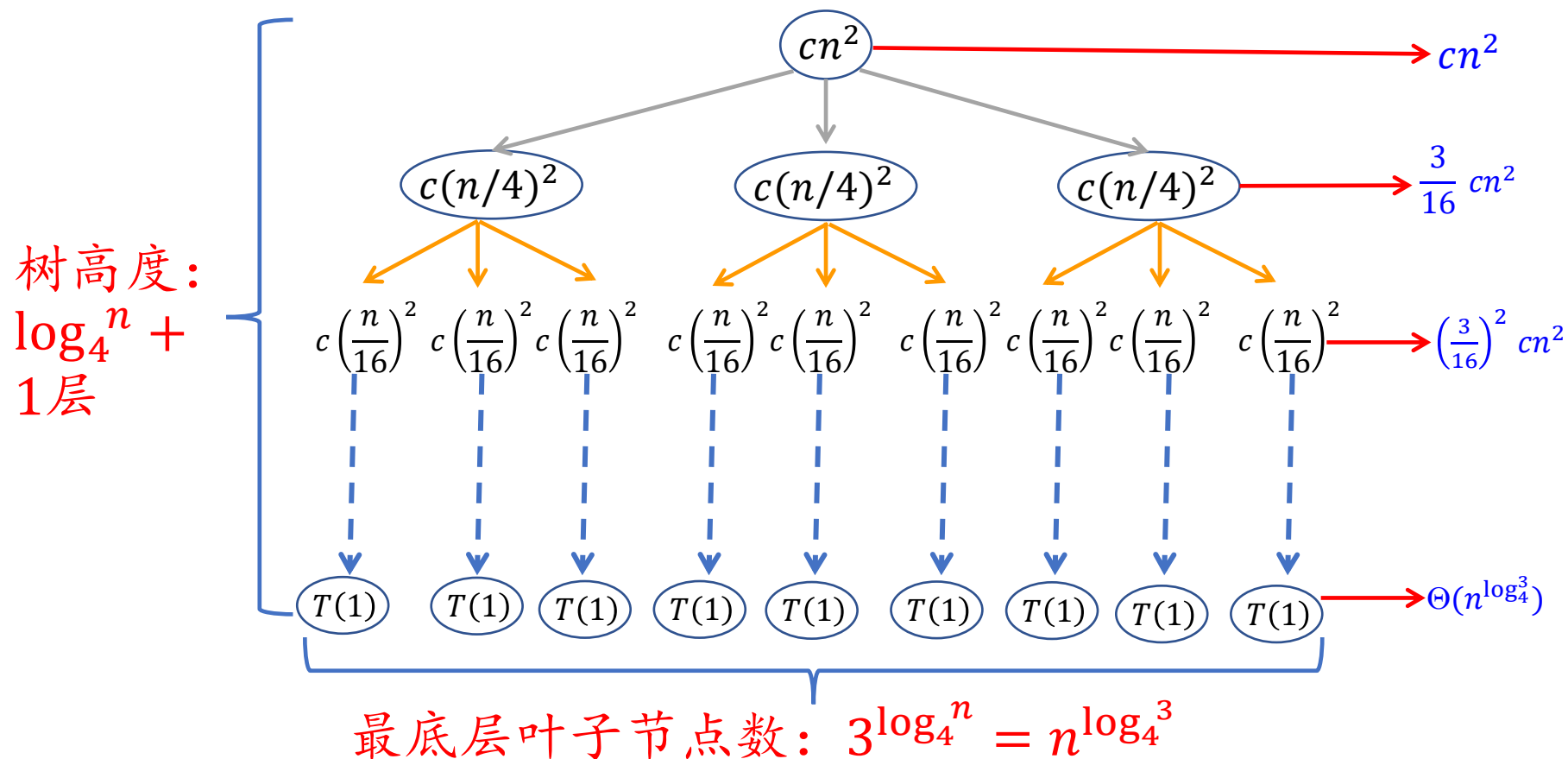
求解递归方程的三个主要方法

- 替换（代入）方法：
 - 首先猜想，
 - 然后用数学归纳法证明。
- 迭代（递归树）方法：
 - 画出递归树，
 - 把方程转化为一个和式，
 - 然后用估计和的方法来求解。
- Master定理方法：
 - 求解型为 $T(n) = a * T\left(\frac{n}{b}\right) + f(n)$ 的递归方程。

这一部分对应于《算法导论》
第三版的4.3-4.5节

递归树

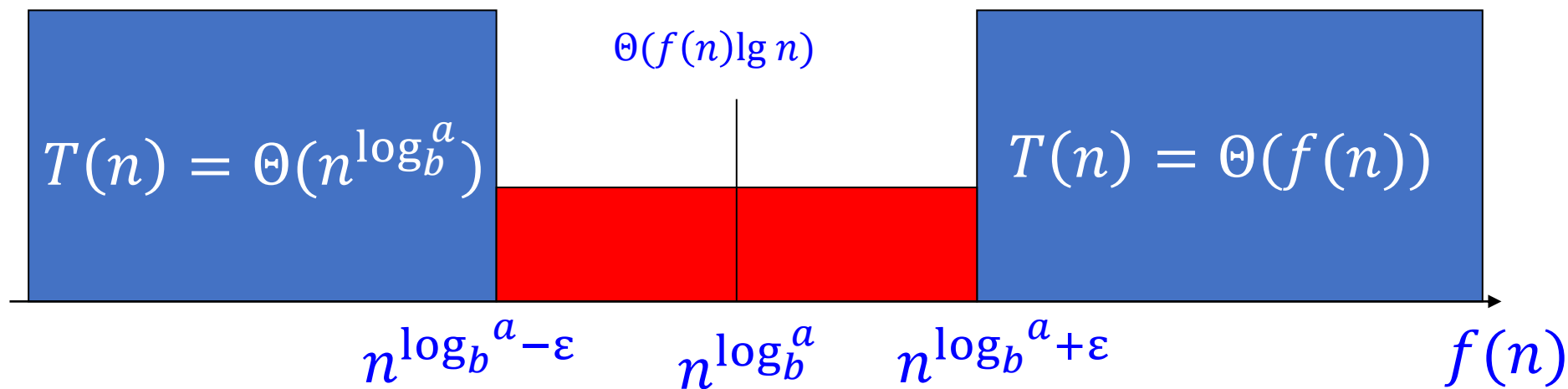
例1. $T(n) = 3 * T\left(\left\lfloor \frac{n}{4} \right\rfloor\right) + \Theta(n^2)$ 。假定 n 是4的幂。



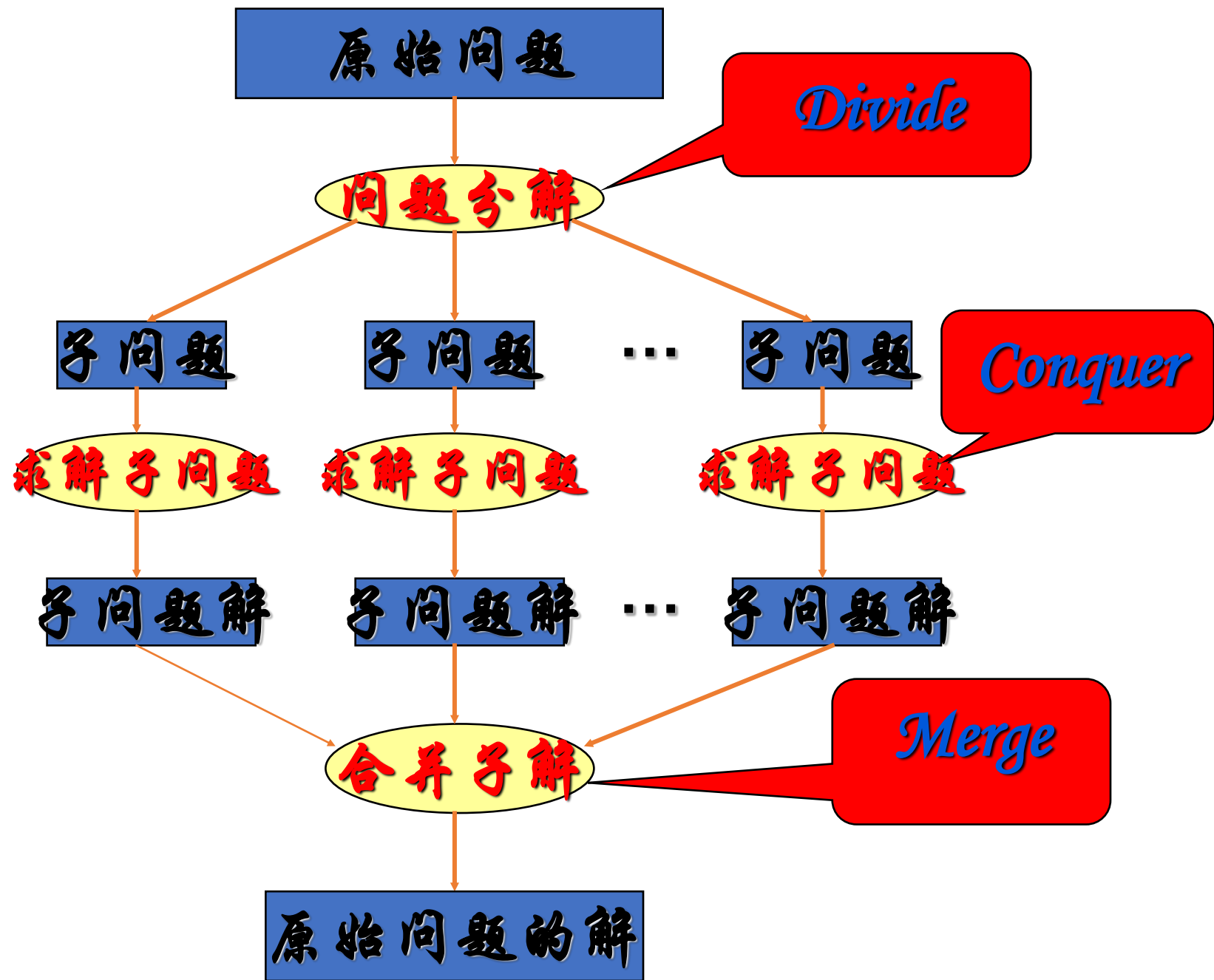
总的代价之和=迭代树右侧所有代价之和!!!

直观地：我们用 $f(n)$ 与 $n^{\log_b a}$ 比较：

- (1) 若 $n^{\log_b a}$ 多项式地大，则 $T(n) = \Theta(n^{\log_b a})$ 。
- (2) 若 $f(n)$ 与 $n^{\log_b a}$ 同阶，则 $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$ 。
- (3) 若 $f(n)$ 多项式地大，则 $T(n) = \Theta(f(n))$ 。



对于红色部分，Master定理无能为力



改进分治算法

$$X = \overset{n/2\text{位}}{\boxed{A}} \overset{n/2\text{位}}{\boxed{B}} \quad Y = \overset{n/2\text{位}}{\boxed{C}} \overset{n/2\text{位}}{\boxed{D}}$$

$$\begin{aligned} XY &= (A2^{n/2} + B)(C2^{n/2} + D) \\ &= AC2^n + (AD+BC)2^{n/2} + BD \\ &= AC2^n + ((A-B)(D-C)+AC+BD)2^{n/2} + BD \end{aligned}$$

算法

1. 计算A-B和D-C;
2. 计算n/2位乘法AC、BD、(A-B)(C-D);
3. 计算(A-B)(D-C)+BC+AD;
4. AC左移n位, ((A-B)(D-C)+AC+BD)左移n/2位;
5. 计算XY

算法复杂性:

$$T(n)=3T(n/2)+\theta(n)$$

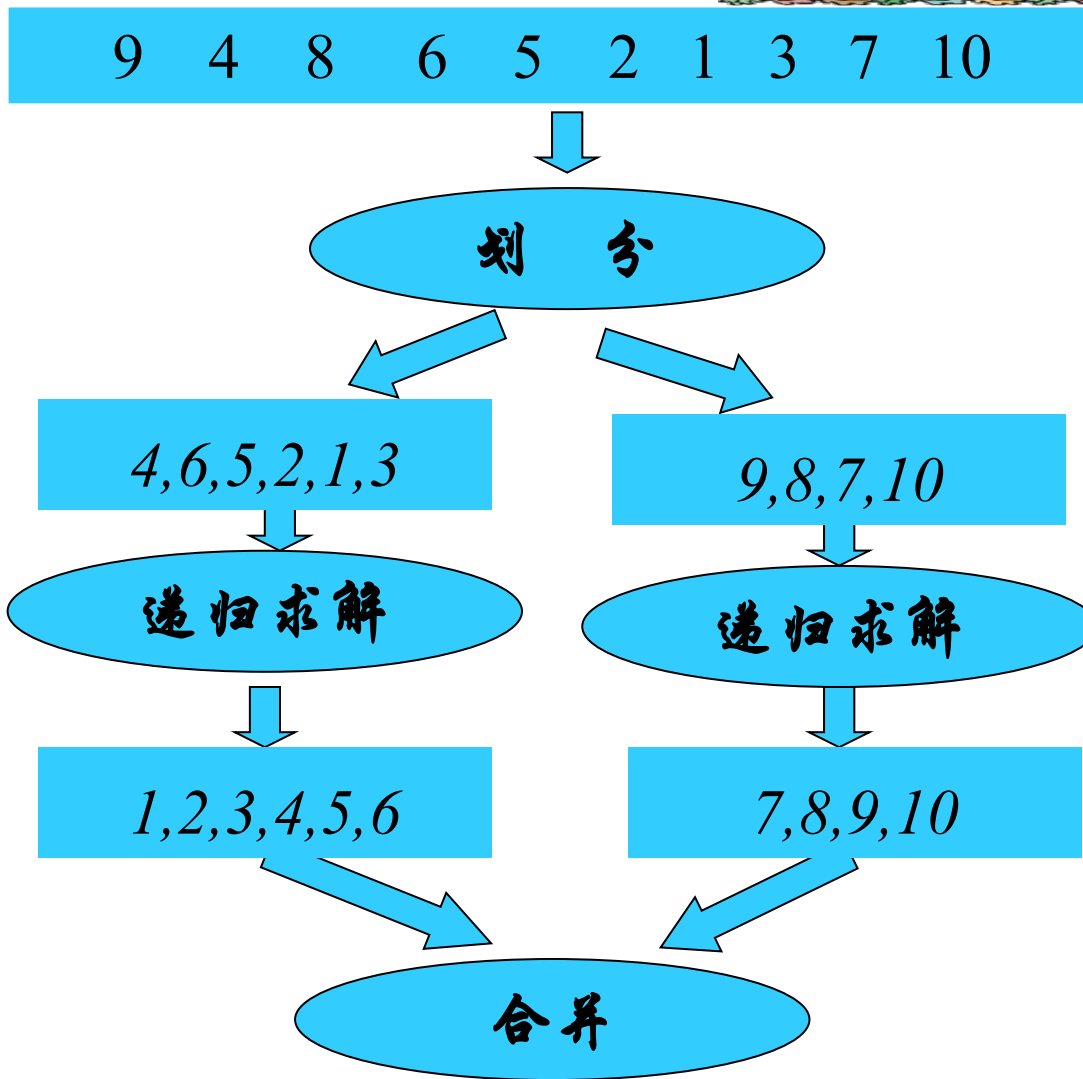
中位数问题定义

Input: 由 n 个数构成的多重集合 X

Output: $x \in X$ 使得 $-1 \leq |\{y \in X \mid y < x\}| - |\{y \in X \mid y > x\}| \leq 1$



基于分治思想的排序算法



划分的策略

1. 选择一个位置将数组划分成两个部分 mergesort

2. 选择一个划分标准 x , 根据元素与 x 的大小关系来划分 quicksort

合并策略

不同的划分策略对应不同的合并策略

由于分治思想涉及到递归调用, 需要关心子问题的最一般形式

在排序问题中, 子问题一般形式就是将 $A[i, \dots, j]$ 中的元素排序

思考题 1

给定长度为 n 的单调不减数列数 $a_0, \dots, a_{n-1}$ 和一个数 k , 在 $O(\log n)$ 的时间复杂度内找到满足 $a_i \geq k$ 条件的最小 i , 写出伪代码

- 动态规划算法的设计步骤
 - 分析优化解的结构
 - 递归地定义最优解的代价
 - 自底向上地计算优化解的代价保存之，并获取构造最优解的信息
 - 根据构造最优解的信息构造优化解

问题的定义

- 输入： $\langle A_1, A_2, \dots, A_n \rangle$, A_i 是矩阵
- 输出： 计算 $A_1 \times A_2 \times \dots \times A_n$ 的最小代价方法

矩阵乘法的代价/复杂性： 乘法的次数

若 A 是 $p \times q$ 矩阵， B 是 $q \times r$ 矩阵， 则 $A \times B$ 的代价是 $O(pqr)$

考虑到所有的 k ，优化解的代价方程为

$$m[i, j] = 0 \quad \text{if } i = j$$

$$m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \}$$

$$\text{if } i < j$$

问题的定义

最长公共子序列 (*LCS*) 问题:

输入: $X = (x_1, x_2, \dots, x_n)$

$Y = (y_1, y_2, \dots, y_m)$

输出: $Z = X$ 与 Y 的最长公共子序列

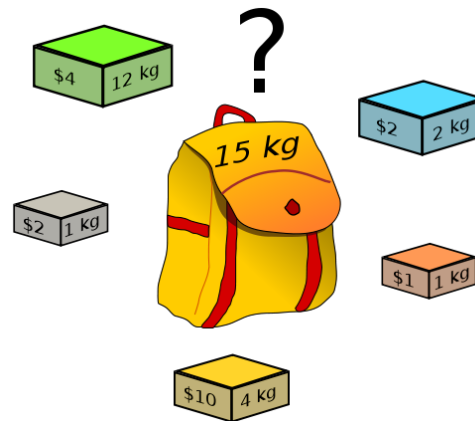
X 和 Y 的 LCS 的优化解结构为

$$LCS_{XY} = LCS_{X_{m-1}Y_{n-1}} + \langle x_m = y_n \rangle \quad \text{if } x_m = y_n$$

$$LCS_{XY} = LCS_{X_{m-1}Y_n} \quad \text{if } x_m \neq y_n, z_k \neq x_m$$

$$LCS_{XY} = LCS_{X_mY_{n-1}} \quad \text{if } x_m \neq y_n, z_k \neq y_n$$

0-1背包问题的定义



给定 n 种物品和一个背包，物品 i 的重量是 w_i ，价值 v_i ，背包容量为 C ，问如何选择装入背包的物品，使装入背包中的物品的总价值最大？

对于每种物品只能选择完全装入或不装入，一个物品至多装入一次。

递归方程

$$m(i, j) = m(i-1, j) \quad 0 \leq j < w_i$$

$$m(i, j) = \max\{m(i-1, j), m(i-1, j-w_i) + v_i\} \quad j \geq w_i$$

初始值

$$m(1, j) = 0 \quad 0 \leq j < w_1$$

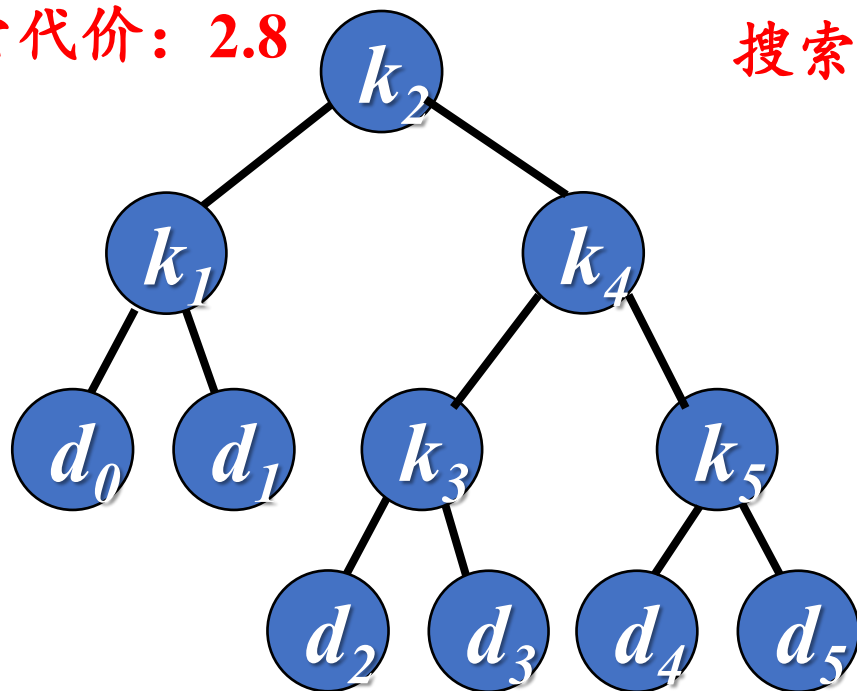
$$m(1, j) = v_1 \quad j \geq w_1$$

最优二叉搜索树 问题的定义

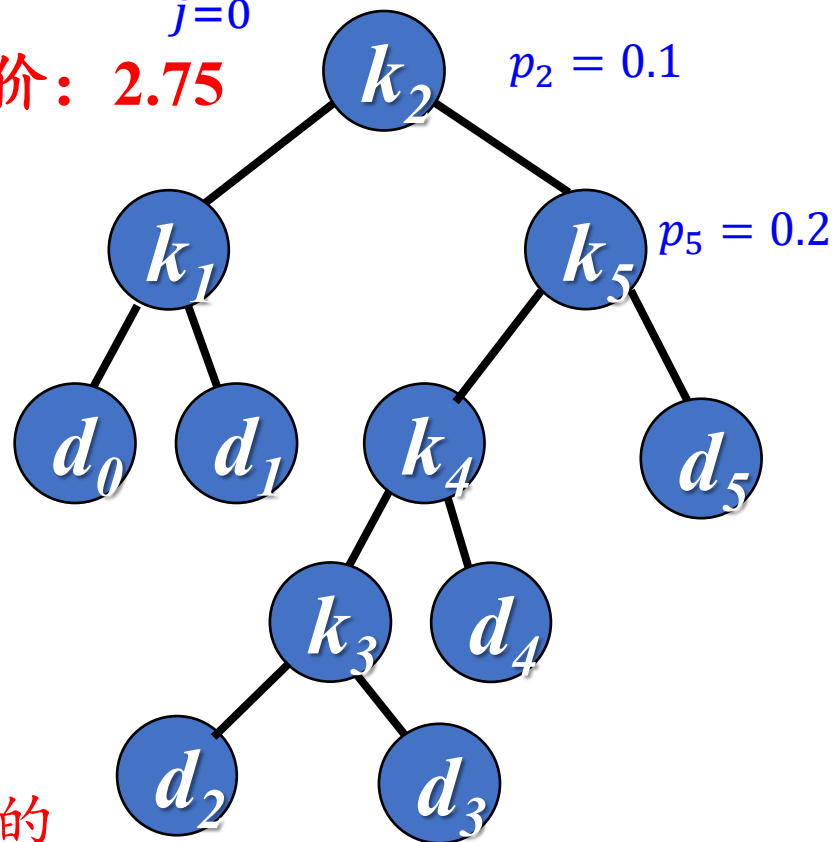
- 搜索树的期望代价

$$E(T) = \sum_{i=1}^n (\text{depth}_T(k_i) + 1)p_i + \sum_{j=0}^n (\text{depth}_T(d_j) + 1)q_j$$

搜索代价: 2.8



搜索代价: 2.75



- 最优二叉搜索树不一定是高度最矮的
- 概率高的关键字不一定出现在二叉搜索树的根结点

自下而上计算优化解的搜索代价

$$q_0 = E(1, 0) \quad E(1, 1) \quad E(1, 2) \quad E(1, 3) \quad E(1, 4)$$

$$q_1 = E(2, 1) \quad E(2, 2) \quad E(2, 3) \quad E(2, 4)$$

$$q_2 = E(3, 2) \quad E(3, 3) \quad E(3, 4)$$

$$q_3 = E(4, 3) \quad E(4, 4)$$

$$q_4 = E(5, 4)$$

矩阵 E 有什么特点?

行号 i 从1到 $n + 1$

列号 j 从0到 n (n 是关键字的个数)

$$E(i, j) = \begin{cases} q_{i-1} = q_j & \text{if } j = i - 1 \\ \min_{i \leq r \leq j} \{E(i, r - 1) + E(r + 1, j) + W(i, j)\} & \text{if } j \geq i \end{cases}$$

贪心算法正确性证明方法



- 证明算法所求解的问题具有优化子结构
- 证明算法所求解的问题具有贪心选择性
- 证明算法确实按照贪心选择性进行局部优化选择

运动安排问题



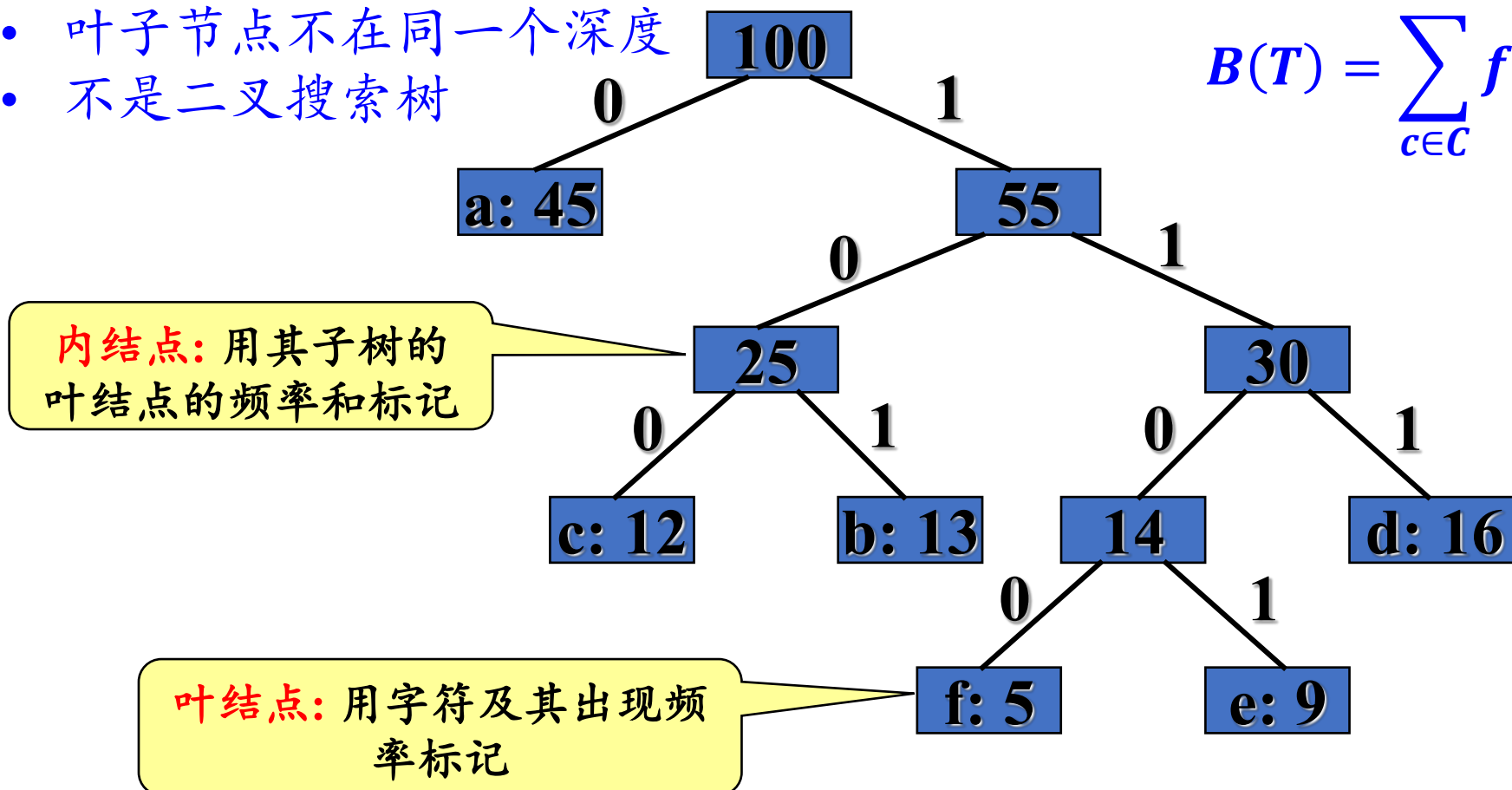
设 $S = \{a_1, a_2, \dots, a_n\}$ 是 n 个活动的集合，各个活动使用同一个资源，资源在同一时间只能为一个活动使用。每个活动 a_i 有起始时间 s_i ，终止时间 f_i ， $s_i < f_i$ 。

问：如何安排活动，能够安排尽量多的活动。最多能安排多少个活动？

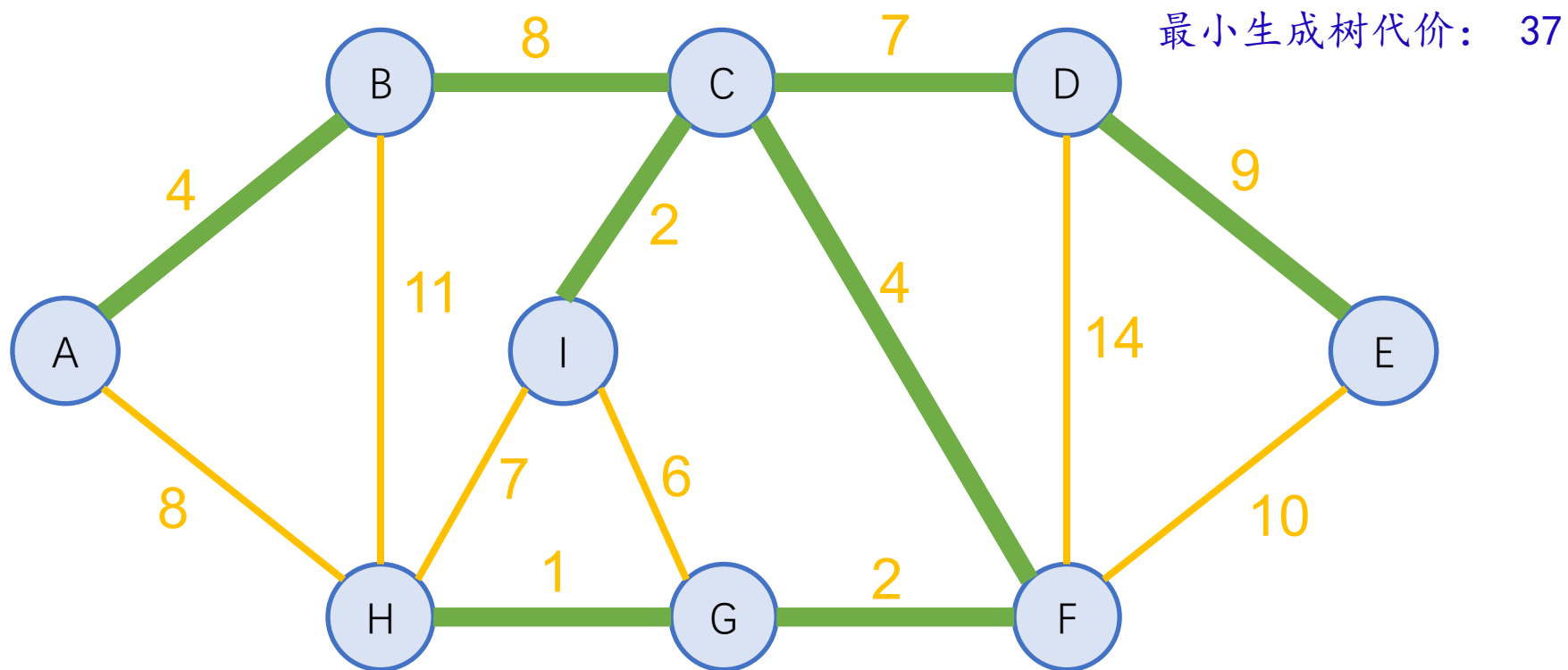
计算机编码

- 前缀编码可以表示成一个二叉树
 - 每个字符是树的叶子
 - 从根结点到叶子结点的路径为对应字符的编码
 - 前缀编码树不唯一
 - 叶子节点不在同一个深度
 - 不是二叉搜索树

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$



问题定义



输入: 一个边加权无向连通图 $G = (V, E)$

输出: 最小代价的生成树 (不唯一)

Kruskal算法

MST-Kruskal(G, W)

Input: 无向连通图 $G = (V, E)$, 权值函数 W

Output: G 的最小生成树

1. $S = \emptyset$; /*初始化为空树*/
2. FOR $\forall v \in V$ DO
3. Make-Set(v); /*创建 $|V|$ 棵树*/
4. 按照 W 权值的非递减顺序排序 E ;
5. FOR $\forall (u, v) \in E$ (按照 W 权值的非递减顺序) DO
6. IF Find-Set(u) $\neq$ Find-Set(v)
 /*检查节点 u, v 是否属于同一棵树 */
7. $S = S \cup \{(u, v)\}$; /*边 (u, v) 加入到集合 S */
8. Union(u, v); /*对节点 u, v 所属的子树进行合并*/
9. Return S .

Prim算法

MST-Prim(G, W, r)

Input: 无向连通图 $G = (V, E)$, 权值函数 W , 初始结点 r

Output: G 的最小生成树

```
1.  FOR  $\forall v \in V$  DO
2.       $\text{key}[v] \leftarrow \infty$ ; /*保存当前联通集合到结点 $v$ 的最小代价*/
3.       $\pi(v) \leftarrow \text{null}$ ; /*结点 $v$ 在联通集合中的父节点*/
4.       $\text{key}[r] \leftarrow 0$ ;      /*保证结点 $r$ 第一个被处理*/
5.       $Q \leftarrow V[G]$ ;      /*最小优先队列*/
6.  WHILE  $Q \neq \emptyset$  DO
7.       $u \leftarrow \text{Extract-Min}(Q)$ ; /*找到 $Q$ 中横跨割  $(V-Q, Q)$  的轻量级边的端点*/
8.      FOR  $\forall v \in \text{Adj}[u]$  DO
9.          IF  $v \in Q$  and  $w(u, v) < \text{key}[v]$  Then
10.              $\pi(v) \leftarrow u$ ;
11.              $\text{key}[v] \leftarrow w(u, v)$ ; /*更新信息*/
12. Return  $A = \{(v, \pi(v)) \mid v \in V - r\}$ .
```

$O(n + n \log n + m \log n) = O(m \log n)$

$O(n \log n + m)$

提纲

6.1 暴力美学：搜索漫谈

6.2 深度优先与广度优先

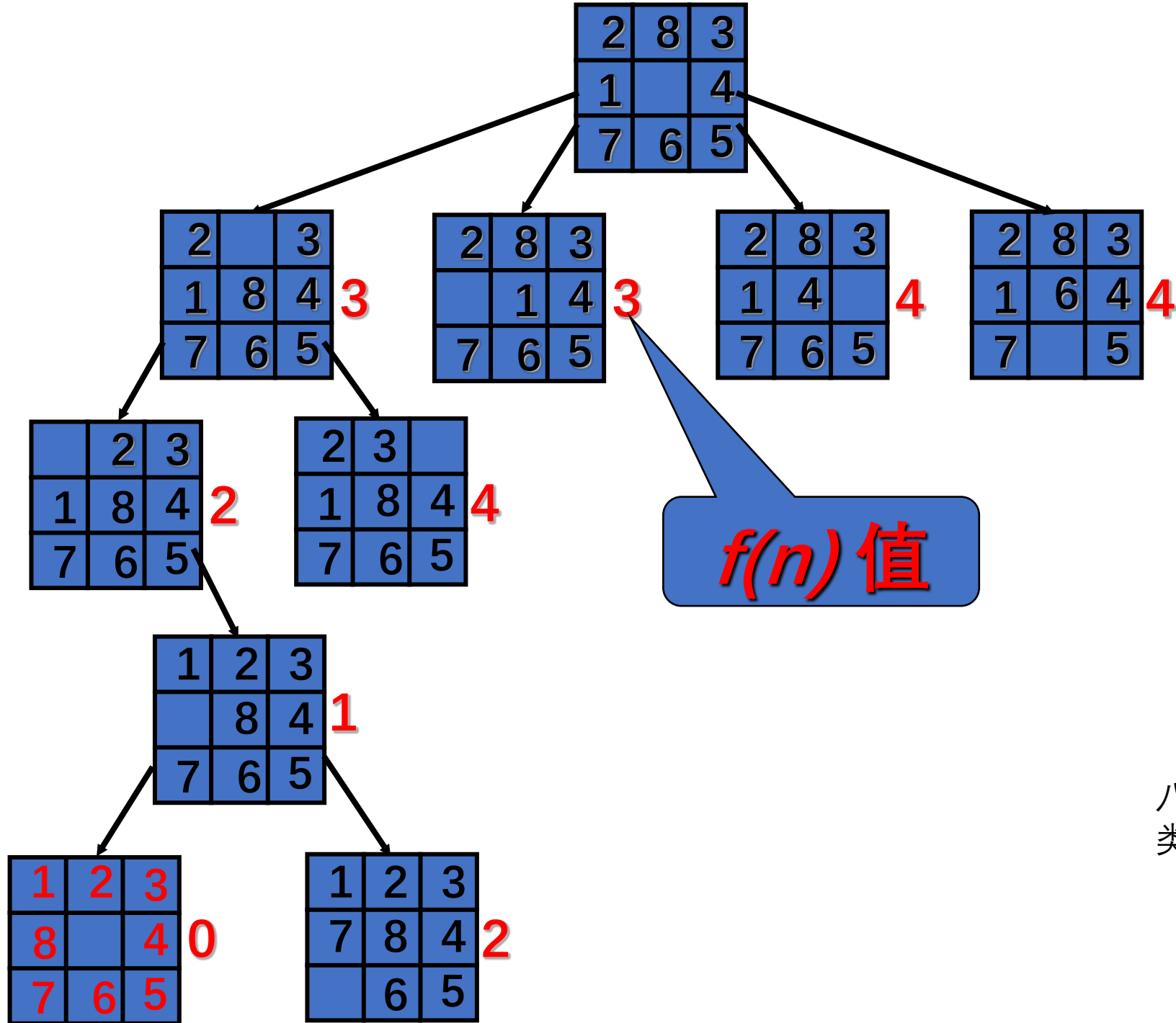
6.3 搜索的优化

6.4 剪枝方法论与人员安排问题

6.5 旅行商问题

6.6 A*算法

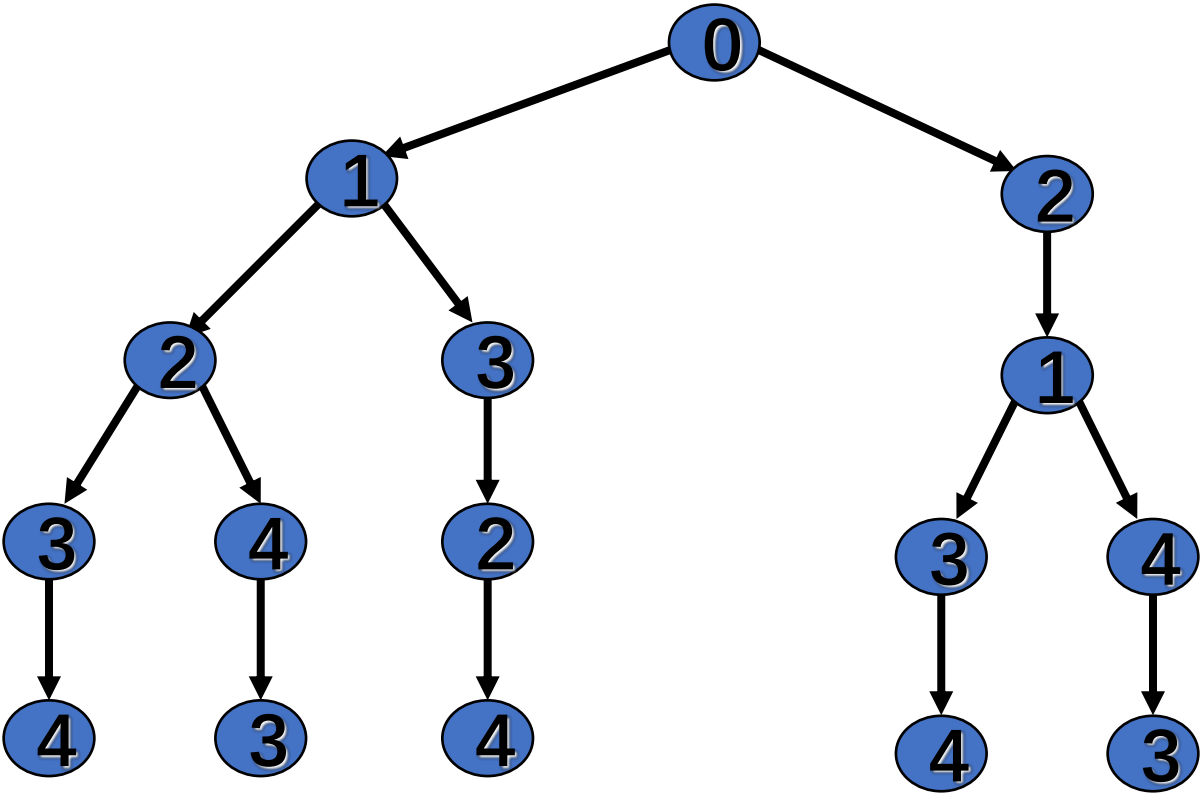
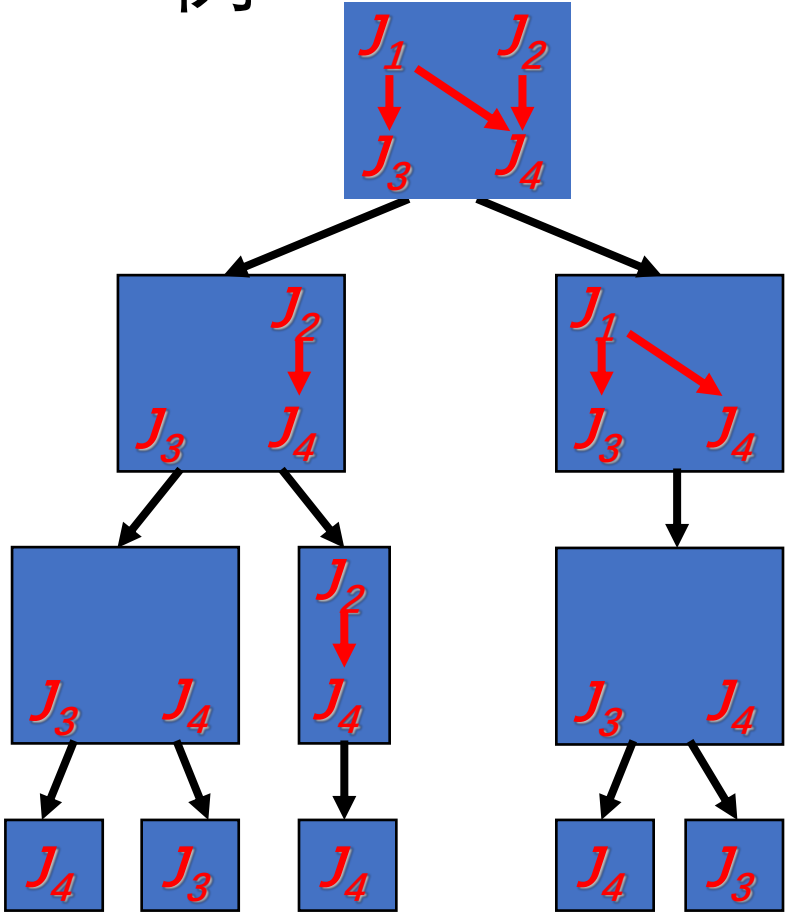




八数码, 棋盘
类游戏

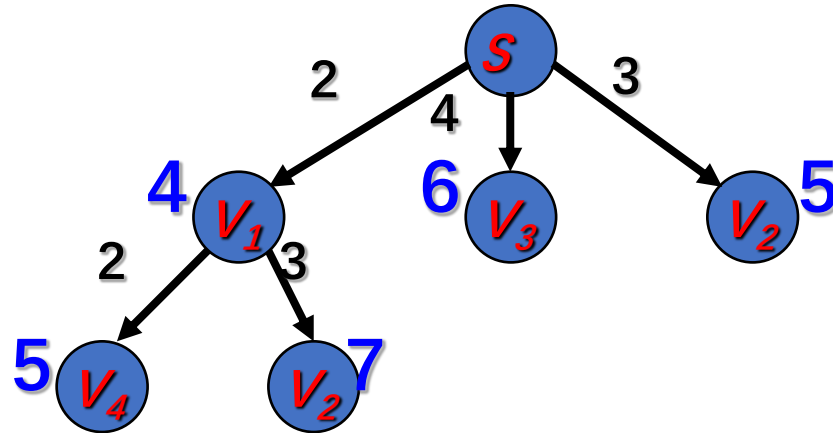
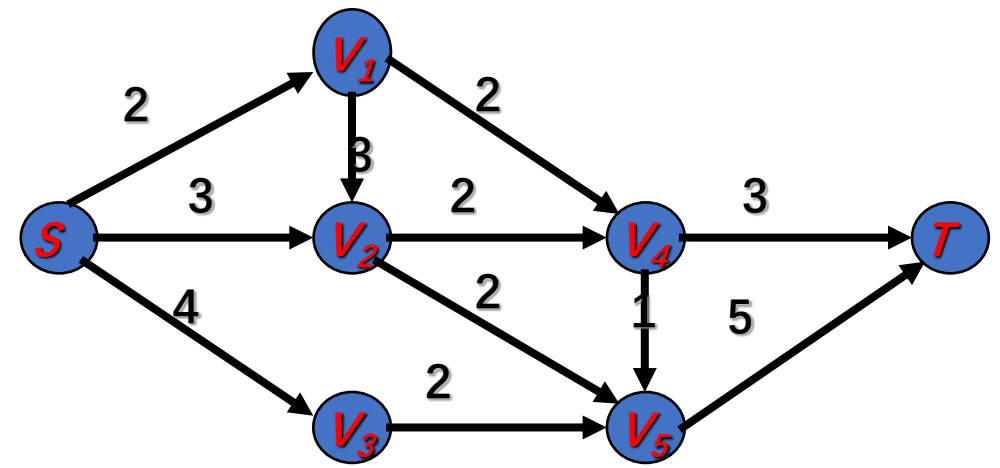
人员安排, 拓扑排序

• 例

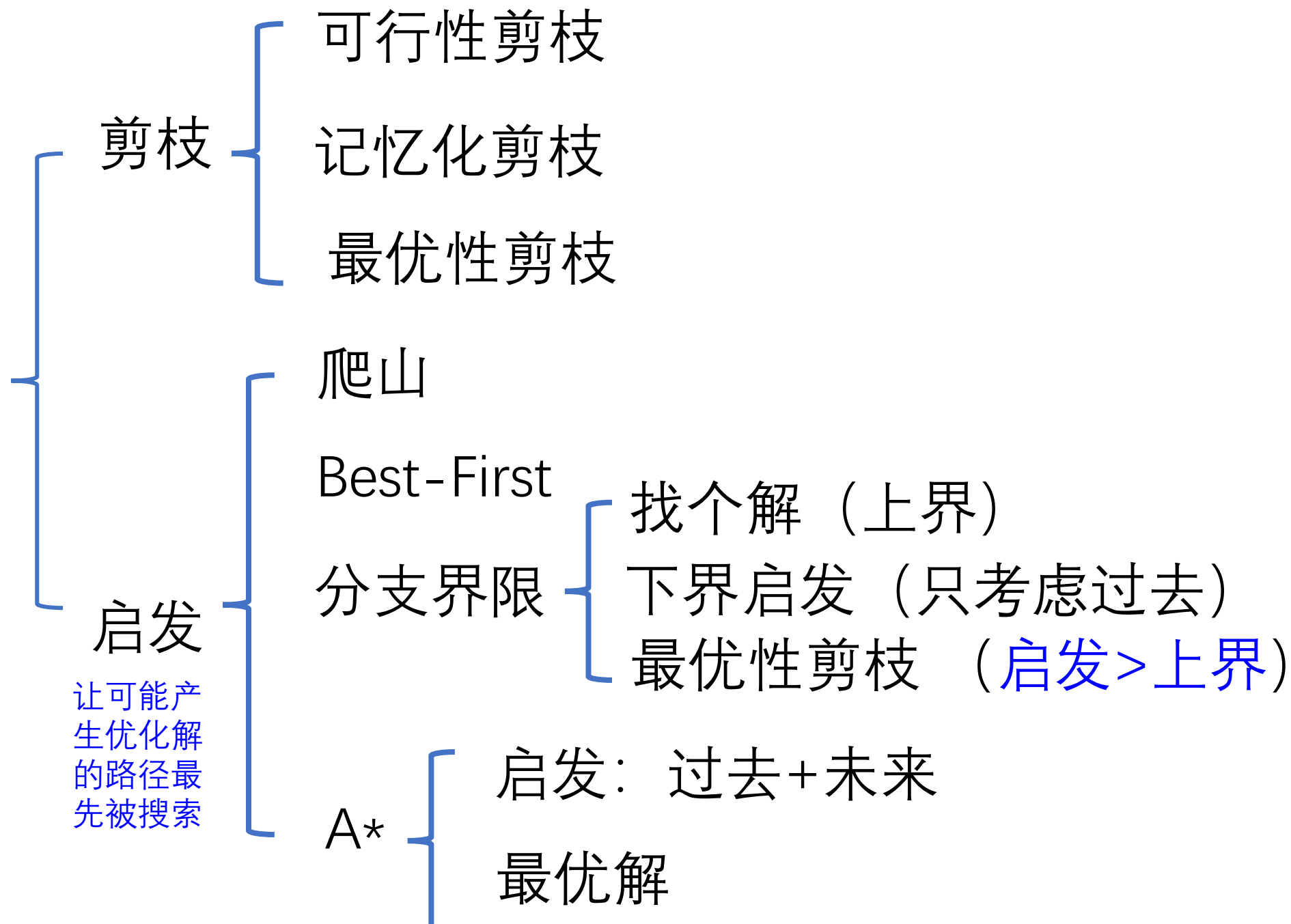




最短路径



$$\begin{aligned}
 g(V_4) &= 2 + 2 = 4 & h(V_4) &= \min\{3, 1\} = 1 & f(V_4) &= 4 + 1 = 5 \\
 g(V_2) &= 2 + 3 = 5 & h(V_2) &= \min\{2, 2\} = 2 & f(V_2) &= 5 + 2 = 7
 \end{aligned}$$



本讲内容

7.1 摊还分析原理

7.2 聚集方法

7.3 会计方法

7.4 势能方法

7.5 动态表操作的摊还分析

本讲内容

8.1 最短路径问题

8.2 网络流问题

8.3 匹配问题

Bellman-Ford 算法

Bellman-Ford(G, s, w)

1. **For** each $v \in V$ **Do**
2. $d[v] \leftarrow \infty$; /*从起点 s 到顶点 v 的路径权值*/
3. $d[s] \leftarrow 0$;
4. **For** $i = 1$ **To** $|V|-1$ **Do**
5. **For** each edge $(u, v) \in E$ **Do**
6. Relax($u, v, w(u, v)$);
7. **For** each edge $(u, v) \in E$ **Do**
8. **If** ($d[v] > d[u] + w(u, v)$) **Then**
9. Return “no solution”;
10. **Return** True

Dijkstra 算法

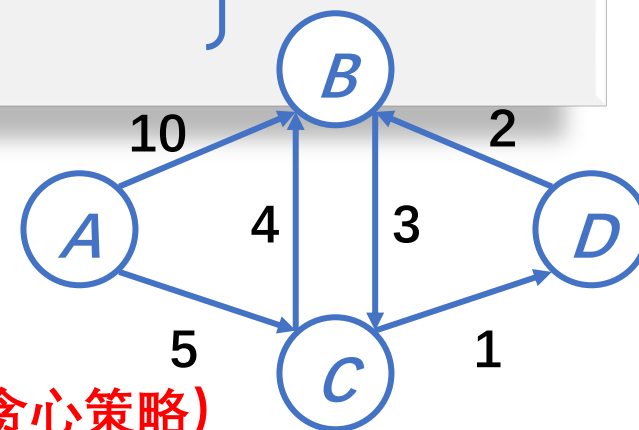
Dijkstra(G,s)

1. For each $v \in V$ Do
2. $d[v] \leftarrow \infty$; /*从起点 s 到顶点 v 的路径权值*/
3. $d[s] \leftarrow 0$; $S \leftarrow \emptyset$; $Q \leftarrow V$; /* S 是已访问结点集合; Q 最小优先队列是未访问结点集合*/
4. While ($Q \neq \emptyset$) Do
5. $u \leftarrow \text{Extract-Min}(Q)$; /* which removes and returns the element with the smallest key from Q */
6. $S \leftarrow S \cup \{u\}$;
7. For $v \in u \rightarrow \text{Adj}()$ Do /*更新与 u 邻接的结点 v */
8. If ($d[v] > d[u] + w(u,v)$) Then
9. $d[v] \leftarrow d[u] + w(u,v)$;

初始化

扩展

松弛步骤



对比Bellman-Ford 算法

- Bellman-Ford 算法对所有的边松弛操作 $|V| - 1$ 次
- Dijkstra算法对所有的边松弛操作一次且仅一次 (贪心策略)

Floyd算法：使用一个D

Floyd(W)

1. $D \leftarrow W$
2. $P \leftarrow 0$
3. For $k \leftarrow 1$ To n Do
4. For $i \leftarrow 1$ To n Do
5. For $j \leftarrow 1$ To n Do
6. If $(D[i,j] > D[i,k] + D[k,j])$ Then
7. $D[i,j] \leftarrow D[i,k] + D[k,j]$
8. $P[i,j] \leftarrow k;$

Ford-Fulkerson 方法

Ford-Fulkerson(G, s, t)

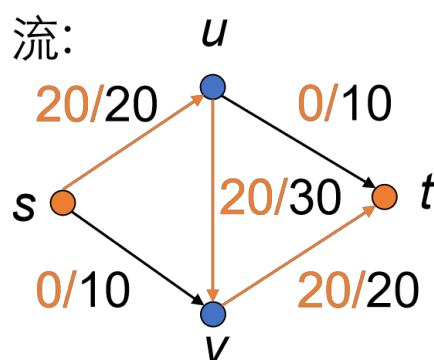
1. **For** each edge (u, v) in $G.E$ **Do**
2. $f(u, v) = 0$
3. **While** there exists a path p from s to t in G_f **Do**
4. $c_f(p) \leftarrow \min\{c_f(u, v) : (u, v) \text{ is in } p\}$
5. **For** each edge (u, v) in p **Do**
6. **If** (u, v) in $G.E$ **Do**
7. $f(u, v) \leftarrow f(u, v) + c_f(p)$
8. **Else** $f(v, u) \leftarrow f(v, u) - c_f(p)$

} 对于所有 e 初始化 $f(e) = 0$

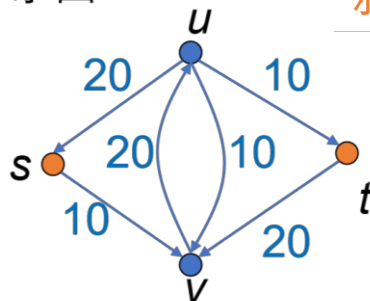
} 找到路径 p 的最小容量

} 沿着路径 p 修改流

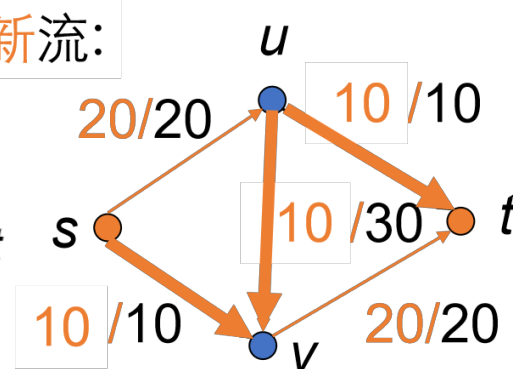
流:



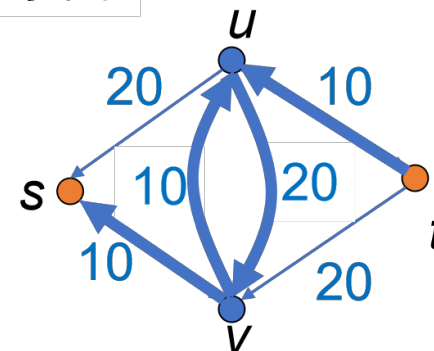
余图:



新流:



新余图:



匈牙利算法

- 匹配 M 是最大匹配，当且仅当对于任意匹配 M' ，有 $|M| \geq |M'|$ 。
- 求二分图的最大匹配 M 。
- 基本步骤：
 1. 置 M 为空；
 2. 找出一条增广路径 P ，通过取反操作，得到更大的匹配；
 3. 重复步骤2，直到找不出增广路为止。

朴素匹配算法

Naive-Search(T,P)

1. $n \leftarrow \text{length}(T);$
2. $m \leftarrow \text{length}(P);$
3. $\text{flag} \leftarrow 1$
4. **For** $s \leftarrow 0$ **to** $n - m$ **Do**
5. $j \leftarrow 0$
6. **While** $T[s+j] = P[j]$ **Do**
7. $j \leftarrow j+1$
8. **If** $j = m$ **Then**
9. print s
10. $\text{flag} \leftarrow 0$
11. **If** flag **Then**
12. **Return** -1

Rabin-Karp算法

Rabin-Karp-Search(T,P)

1. $q \leftarrow a$ /*q取为素数*/
2. $c \leftarrow 10^{m-1} \bmod q$
3. $fp \leftarrow 0, ft \leftarrow 0, \text{flag} \leftarrow 1$
4. For $i = 0$ To $m - 1$ Do // preprocessing
5. $fp \leftarrow (10 * fp + P[i]) \bmod q$
6. $ft \leftarrow (10 * ft + T[i]) \bmod q$
7. For $s = 0$ To $n - m$ Do // matching
8. If $fp = ft$ Then //run a loop to compare strings
9. If $P[0..m-1] = T[s..s+m-1]$ Then
10. print s
11. flag $\leftarrow 0$
12. $ft \leftarrow ((ft - T[s] * c) * 10 + T[s+m]) \bmod q$
13. If flag Then
14. Return -1

套用mod q运算
公式1,2得到



有限状态自动机 字符串匹配算法

构造状态转移表

Compute-Transition-Table(P, S)

1. $m \leftarrow P.length$ /*模式串P的长度*/
2. for $q = 0$ to m /*q已匹配字符个数*/
3. for each character x in S /*考虑每个可能加入的字母*/
4. $k \leftarrow \min(m, q+1)$
5. repeat
6. $k \leftarrow k-1$
7. until **$P[0:k]$ is the suffix of $P[0:q-1]+'x'$**
8. $\delta(q, 'x') \leftarrow k+1$
9. return δ

每次状态变化, $\delta(q, x)$ 最大是 $(q + 1)$ 或者 m

最坏时间复杂度 $O(m^3|\Sigma|)$, 可以改进为 $O(m|\Sigma|)$

有限状态自动机字符串匹配算法

Finite-Automation-Matcher(T, P, Σ)

```
1.   $n \leftarrow T.length$ 
2.   $q \leftarrow 0, flag \leftarrow 1$ 
3.   $\delta \leftarrow \text{Compute-Transition-Table}(P, S) /*构造状态转移表*/$ 
4.  For  $i = 0$  To  $n$  Do  $/*对主串T的每个字符*/$ 
5.       $q \leftarrow \delta(q, T[i])$ 
6.      If  $q = m$  Then
7.          Print "pattern occurs with shift"  $i - m + 1$            $/* i - m + 1$ 匹配开始的下标
8.           $*/$ 
9.       $flag \leftarrow 0$ 
10. If  $flag$  Then
11.     Return  $-1$ 
```

- 分析:

- 匹配阶段: $O(n)$

- 内存过多: $O(m|\Sigma|)$, 过多的预处理 $O(m^3|\Sigma|)$



KMP 算法

预先构造前缀表 π

Compute-Prefix(P)

1. $m \leftarrow P.length$
2. Let $\pi[0,...,m]$ be a new array
3. $\pi[0] \leftarrow 0, \pi[1] \leftarrow 0$
4. $k \leftarrow 0$
5. For $q=2$ To m Do
6. While $k>0$ and $P[k] \neq P[q-1]$
7. $k = \pi[k]$
8. If $P[k] == P[q-1]$
9. $k \leftarrow k+1$
10. $\pi[q] \leftarrow k$
11. Return π

q : 模式串 P 与主字符串 T 中已匹配上的字符个数

时间复杂度 $\Theta(m)$

Knuth-Morris-Pratt 算法

KMP-Search(T,P)

1. $m \leftarrow P.length$
2. $n \leftarrow T.length, flag \leftarrow 1$
3. $\pi \leftarrow \text{Compute-Prefix}(P)$
4. $q \leftarrow 0$ // number of characters matched
5. For $i = 0$ To $n-1$ // scan the text from left to right
6. While $q > 0$ and $P[q] \neq T[i]$ // next character does not match
7. $q \leftarrow \pi[q]$
8. If $P[q] = T[i]$ // next character matches
9. $q \leftarrow q + 1$
10. If $q = m$ // all P's characters matched
11. Print "pattern occurs with shift" $i-m+1$,
12. $flag \leftarrow 0$
13. $q \leftarrow \pi[q]$
14. If $flag$ Then
15. Return -1

Boyer-Moore-Horspool 算法

BMH-Search(T,P)

```
1.  flag ← 1
2.      // compute the shift table for P
3.  For c = 0 To |S|
4.      shift[c] = m
5.  For k = 0 To m-2
6.      shift[P[k]] = m-1-k
7.  // search
8.  s ← 0
9.  While s ≤ n-m Do
10.     j ← m-1 /*从模式串P的后面往前匹配*/ start from the end
11.     // check if T[s..s + m-1] = P[0..m-1]
12.     While T[s+j] = P[j] Do
13.         j ← j-1
14.     If j < 0 Then
15.         Print "pattern occurs with shift" s
16.         flag ← 0
17.         s ← s + shift[T[s + m-1]] // shift by last letter
18. If flag Then
19.     return -1
```

} 计算偏移表